

EXPLORATORY DATA ANALYSIS

USING SQL TO CLEAN, TRANSFORM AND ANALYSE

This project focuses on a company dealing in classic, vintage, and collector automobiles. The dataset, MintClassics, contains information on sales, products, services, and operational processes.





The first step is to explore and extract our database which is done by uploading the MintClassics database file into MySQL Workbench. The data was further explored and exported to Excel to identify clean, remove duplicates and mismatches and extract the relevant information from the dataset. The selected data was then imported into a CSV file for further processing.

This section includes a screenshot of the SQL command used to extract the required data.

Limit to 1000 rows

```

1 • SELECT
2     c.contactFirstName, -- Matches 'contactFullName' if concatenated
3     c.customerName,
4     c.country,
5     o.orderNumber,
6     o.orderDate,
7     p.productName,
8     c.salesRepEmployeeNumber, -- Matches 'salesRep'
9     c.creditLimit,
10    od.quantityOrdered,
11    od.priceEach,
12    (od.quantityOrdered * od.priceEach) AS TotalAmount
13 FROM customers c
14 JOIN orders o ON c.customerNumber = o.customerNumber
15 JOIN orderdetails od ON o.orderNumber = od.orderNumber
16 JOIN products p ON od.productCode = p.productCode
17 ORDER BY o.orderNumber;

```

Result Grid

	contactFirstName	customerName	country	orderNumber	orderDate	productName
▶	Dorothy	Online Diecast Creations Co.	USA	10100	2003-01-06	1917 Grand Touring Sedan
	Dorothy	Online Diecast Creations Co.	USA	10100	2003-01-06	1911 Ford Town Car
	Dorothy	Online Diecast Creations Co.	USA	10100	2003-01-06	1932 Alfa Romeo 8C2300 Spic
	Dorothy	Online Diecast Creations Co.	USA	10100	2003-01-06	1936 Mercedes Benz 500k Ro

Result Grid

A MySQL database was then created, and the cleaned data was imported into MySQL Workbench. Although the dataset was already cleaned in Excel, SQL was still used for further data cleaning and transformation as a form of the ETL (Extract, Transform, Load) process before analysis.

	A	B	C	D	E	F	G	H	I	J	K	L
	contactFullName	customerName	country	orderNumber	orderDate	productName	salesRepEmpl	creditLimit	quantityOrdered	priceEach	TotalAmount	
1												
2	Dorothy Young	Online Diecast Cre USA		10100	1/6/2003	1917 Grand Touring Sedan	1216	114200	30	136	4080	
3	Dorothy Young	Online Diecast Cre USA		10100	1/6/2003	1911 Ford Town Car	1216	114200	50	55.09	2754.5	
4	Dorothy Young	Online Diecast Cre USA		10100	1/6/2003	1932 Alfa Romeo 8C2300 Sp	1216	114200	22	75.46	1660.12	
5	Dorothy Young	Online Diecast Cre USA		10100	1/6/2003	1936 Mercedes Benz 500k f	1216	114200	49	35.29	1729.21	
6	Roland Keitel	Blauer See Auto, C Germany		10101	1/9/2003	1932 Model A Ford J-Coupe	1504	59700	25	108.06	2701.5	
7	Roland Keitel	Blauer See Auto, C Germany		10101	1/9/2003	1928 Mercedes-Benz SSK	1504	59700	26	167.06	4343.56	
8	Roland Keitel	Blauer See Auto, C Germany		10101	1/9/2003	1939 Chevrolet Deluxe Cou	1504	59700	45	32.53	1463.85	
9	Roland Keitel	Blauer See Auto, C Germany		10101	1/9/2003	1938 Cadillac V-16 Preside	1504	59700	46	44.35	2040.1	
10	Michael Frick	Vitachrome Inc., USA		10102	1/10/2003	1937 Lincoln Bertline	1286	76400	39	95.55	3726.45	
11	Michael Frick	Vitachrome Inc., USA		10102	1/10/2003	1936 Mercedes-Benz 500K	1286	76400	41	43.13	1768.33	
12	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1952 Alpine Renault 1300	1504	81700	26	214.3	5571.8	
13	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1962 Lancia Delta 16V	1504	81700	42	119.67	5026.14	
14	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1958 Setra Bus	1504	81700	27	121.64	3284.28	
15	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1940 Ford Pickup Truck	1504	81700	35	94.5	3307.5	
16	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1926 Ford Fire Engine	1504	81700	22	58.34	1283.48	
17	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1913 Ford Model T Speedst	1504	81700	27	92.19	2489.13	
18	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1934 Ford V8 Coupe	1504	81700	35	61.84	2164.4	
19	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	18th Century Vintage Horse	1504	81700	25	86.92	2173	
20	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1917 Maxwell Touring Car	1504	81700	46	86.31	3970.26	
21	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1940s Ford truck	1504	81700	36	98.07	3530.52	
22	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1939 Cadillac Limousine	1504	81700	41	40.75	1670.75	
23	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1962 Volkswagen Microbus	1504	81700	36	107.34	3864.24	
24	Jonas Bergulfsen	Baane Mini Import: Norway		10103	1/29/2003	1936 Chrysler Airflow	1504	81700	25	88.62	2215.5	

mintclassics_customer3

Here are the primary types of data cleaning and transformation tasks typically performed using SQL in this project:

Typical Transformation Workflow for MintClassics:

Type used	SQL Operation Command Used	Purpose
Cleaning	WHERE, COALESCE, UPPER	Removes noise and standardizes inputs.
Transformation	SUM, AVG, CASE	Creates KPIs like "Revenue" or "Inventory Turnover."
Structuring	JOIN, UNION, WITH (CTEs)	Combines disparate tables into one cohesive report.

Example SQL does not understand this date format 1/6/2003 therefore transformed into Year- Month-Date, thou day was excluded in the queries used in creating in Analysis 7.

You can check it for better understanding

	salesYear	salesMonth	totalRevenue
►	2003	January	116692.77
	2003	February	128403.64
	2003	March	160517.14
	2003	April	185848.59
	2003	May	179435.55
	2003	June	150470.77
	2003	July	201940.36
	2003	August	178257.11
	2003	September	236697.85
	2003	October	514336.21
	2003	...	292385.21

ANALYSIS 1: TOTAL PRODUCTS ORDERED

Using these SQL queries with SUM, total Products ordered were calculated

The screenshot shows the SQL Developer interface. On the left, the 'SCHEMAS' pane displays the 'mintclassics' schema with tables like 'customers', 'employees', and 'orderdetails'. The main editor window contains the following SQL query:

```
-- 1. TOTAL NUMBER OF PRODUCTS ORDERED
-- =====
-- Execute the selected portion of the script or everything, if there is no selection
-- =====
10 • SELECT SUM(quantityOrdered) AS totalItemsSold
11 FROM orderdetails;
12
13 -- =====
```

Below the query editor, the 'Result Grid' shows the execution results:

totalItemsSold
105516

The right sidebar contains buttons for 'Result Grid' and 'Form Editor'.

ANALYSIS 2: TOTAL REVENUE

Calculate the total revenue using an SQL query.

The screenshot shows the SQL Developer interface. The main editor window contains the following SQL query:

```
14 -- 2. TOTAL REVENUE
15 -- =====
16 • SELECT
17 SUM(quantityOrdered * priceEach) AS totalRevenue
18 FROM orderdetails;
19
20
```

Below the query editor, the 'Result Grid' shows the execution results:

totalRevenue
9604190.61

The right sidebar contains buttons for 'Result Grid' and 'Form Editor'.

ANALYSIS 3: COUNTRIES WITH HIGHEST SALES

With SQL queries using LEFT JOIN, GROUPBY, Countries with highest Sales limited to 10.

This analysis is based on aggregated sales data, ensuring accurate comparison across countries.”

```
22  -- =====
23  -- 4. COUNTRY SALES PERFORMANCE
24  -- =====
25  • SELECT
26      c.country,
27      SUM(od.quantityOrdered * od.priceEach) AS totalRevenue
28  FROM customers c
29  JOIN orders o ON c.customerNumber = o.customerNumber
30  JOIN orderdetails od ON o.orderNumber = od.orderNumber
31  GROUP BY c.country
32  LIMIT 10;
```

Result Grid

	country	totalRevenue
▶	France	1007374.02
	USA	3273280.05
	Australia	562582.59
	Norway	270846.30
	Germany	196470.99
	Spain	1099389.09
	Sweden	187638.35
	Denmark	218994.92

Result 78 Result 79 Result 80 Result 81 × Result 82 Result 83 Result 84 Result 85 Read Only

ANALYSIS 4: STOCK IN WAREHOUSE

Understanding Products quantity in Stock across warehouse

The SQL commands used are structuring, **JOIN**, **FROM**, and **ORDERBY**.

This insight supports better demand planning, inventory balancing, and cost control strategies especially in the case of warehouse utilization

```
36  -- 5. STOCK IN WAREHOUSE
37  -- =====
38  •  SELECT
39      w.warehouseName,
40      p.productName,
41      p.quantityInStock
42  FROM products p
43  JOIN warehouses w
44      ON p.warehouseCode = w.warehouseCode
45  ORDER BY p.quantityInStock ASC
46  LIMIT 10;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:	Fetch rows:	Result Grid	Form Editor	Field Types
warehouseName	productName	quantityInStock					
North	1960 BSA Gold Star DBD34	15					
East	1968 Ford Mustang	68					
West	1928 Ford Phaeton Deluxe	136					
North	1997 BMW F650 ST	178					
South	Pont Yacht	414					
West	1911 Ford Town Car	540					
West	1928 Mercedes-Benz SSK	548					
North	F/A 18 Hornet 1/72	551					
North	2002 Yamaha YZR M1	600					
South	The Mayflower	737					

ANALYSIS 5: SALES REPRESENTATIVE PERFORMANCE

This insight enables stakeholders to evaluate individual performance, recognize high achievers, and design incentive programs that drive motivation and productivity. Additionally, it helps identify performance gaps and opportunities for training and development limited to 5 Sales Representative.

```
49  -- 6. SALESREP PERFORMANCE
50  -- =====
51  •  SELECT
52      e.firstName,
53      e.lastName,
54      e.jobTitle,
55      SUM(od.quantityOrdered * od.priceEach) AS totalSales
56  FROM employees e
57  JOIN customers c ON e.employeeNumber = c.salesRepEmployeeNumber
58  JOIN orders o ON c.customerNumber = o.customerNumber
59  JOIN orderdetails od ON o.orderNumber = od.orderNumber
60  GROUP BY e.employeeNumber
61  ORDER BY totalSales DESC
62  LIMIT 5;
```

Result Grid

	firstName	lastName	jobTitle	totalSales
▶	Gerard	Hernandez	Sales Rep	1258577.81
	Leslie	Jennings	Sales Rep	1081530.54
	Pamela	Castillo	Sales Rep	868220.55
	Larry	Bott	Sales Rep	732096.79
	Barry	Jones	Sales Rep	704853.91

Result Grid

Form Editor

ANALYSIS 6: PRODUCT HIGHEST DEMAND

Using SQL queries such as SUM, JOIN, GROUP BY, and ORDER BY, I analyzed the seven most in-demand products based on revenue. This analysis provides stakeholders with clear insights into the best-performing products as well as those with lower demand, enabling the team to make more informed strategic decisions aimed at improving underperforming products.

```
/4      -- =====
75      -- 7. PRODUCT HIGEST DEMAND
76      -- =====
77      • SELECT
78          p.productName,
79          SUM(od.quantityOrdered) AS totalQuantitySold,
80          SUM(od.quantityOrdered * od.priceEach) AS totalRevenue -- This shows the dollar value
81      FROM products p
82      JOIN orderdetails od ON p.productCode = od.productCode
83      GROUP BY p.productName
84      ORDER BY totalQuantitySold DESC
85      LIMIT 7;
```

productName	totalQuantitySold	totalRevenue
1992 Ferrari 360 Spider red	1808	276839.98
1937 Lincoln Berline	1111	102563.52
American Airlines: MD-11S	1085	71753.93
1941 Chevrolet Special Deluxe Cabriolet	1076	102537.45
1930 Buick Marquette Phaeton	1074	41599.24
1940s Ford truck	1061	114232.79
1969 Harley Davidson Ultimate Chopper	1057	90157.77

ANALYSIS 7: SALES TREND

SQL (Structured Query Language) was used to efficiently extract and analyze sales data from the Mint Classics dataset for the period 2003 to 2005. By aggregating sales on a yearly basis, this analysis provides a clear view of revenue trends over time, enabling stakeholders to assess business performance and identify growth or decline patterns.

The screenshot displays a SQL IDE interface with a query editor, a result grid, and an execution log. The query editor shows a SQL query that aggregates sales data by year and month for the period 2003 to 2005. The result grid shows the output of the query, which includes columns for salesYear, salesMonth, and totalRevenue. The execution log shows the steps of the query execution, including the time taken for each step and the number of rows returned.

```
1 SELECT
2   YEAR(o.orderDate) AS salesYear,
3   MONTHNAME(o.orderDate) AS salesMonth,
4   SUM(od.quantityOrdered * od.priceEach) AS totalRevenue
5 FROM orders o
6 JOIN orderdetails od ON o.orderNumber = od.orderNumber
7 WHERE YEAR(o.orderDate) BETWEEN 2003 AND 2005
8 GROUP BY salesYear, salesMonth
```

salesYear	salesMonth	totalRevenue
2003	January	116692.77
2003	February	128403.64
2003	March	160517.14
2003	April	185848.59
2003	May	179435.55
2003	June	150470.77

#	Time	Action	Message	Duration / Fetch
3	07:42:08	SELECT	COUNT(DISTINCT orderNumber) AS TotalNumberOfOrders. SUM(q...	1 row(s) returned 0.015 sec / 0.000 sec
4	07:42:17	SELECT	COUNT(DISTINCT orderNumber) AS TotalNumberOfOrders. SUM(q...	1 row(s) returned 0.016 sec / 0.000 sec
5	07:42:59	SELECT	YEAR(o.orderDate) AS salesYear. MONTHNAME(o.orderDate) AS ...	29 row(s) returned 0.031 sec / 0.000 sec
6	07:43:45	SELECT	c.country, SUM(od.quantityOrdered * od.priceEach) AS TotalReven...	10 row(s) returned 0.000 sec / 0.000 sec
7	07:44:38	SELECT	w.warehouseName, p.productName, p.quantityInStock, SUM(o...	4 row(s) returned 0.281 sec / 0.000 sec
8	08:09:43	SELECT	w.warehouseName, p.productName, p.quantityInStock FROM pr...	1 row(s) returned 0.000 sec / 0.000 sec

ANALYSIS 8: CREDIT LIMIT

The SQL query was used to analyze and understand customers' credit limits. If credit limits are not properly regulated, they can negatively impact the company's revenue. In this analysis, customers' credit limits were limited to 10 for visualization.

```

91 • SELECT
92     c.customerName,
93     c.creditLimit,
94     SUM(od.quantityOrdered * od.priceEach) AS totalOutstandingValue,
95     (c.creditLimit - SUM(od.quantityOrdered * od.priceEach)) AS remainingCredit
96 FROM customers c
97 JOIN orders o ON c.customerNumber = o.customerNumber
98 JOIN orderdetails od ON o.orderNumber = od.orderNumber
99 GROUP BY c.customerName, c.creditLimit
100 HAVING totalOutstandingValue > (c.creditLimit * 0.8) -- Flag if over 80% used
101 ORDER BY remainingCredit ASC
102 LIMIT 10;

```

customerName	creditLimit	totalOutstandingValue	remainingCredit
Euro+ Shopping Channel	227600.00	820689.54	-593089.54
Mini Gifts Distributors Ltd.	210500.00	591827.34	-381327.34
Down Under Souvenirs, Inc	88000.00	154622.08	-66622.08
The Sharp Gifts Warehouse	77600.00	143536.27	-65936.27
Salzburg Collectables	71700.00	137480.07	-65780.07
Australian Collectors, Co.	117300.00	180585.07	-63285.07
Dragon Souvenirs, Ltd.	103800.00	156251.03	-52451.03
Reims Collectables	81100.00	126983.19	-45883.19
Danish Wholesale Imports	83400.00	129085.12	-45685.12
Gifts4AllAges.com	41900.00	84340.32	-42440.32

BUSINESS INSIGHTS:

For more detailed insights, the transformed SQL results were exported into a dashboard, as visual representation provides a clearer understanding of complex data. This dashboard offers a simplified yet comprehensive view of the overall project through KPIs and charts, including revenue, warehouse stock levels, highest product demand, sales representative performance and country with the highest sales.

Despite these achievements, **the company is experiencing a downward trend, which is more clearly highlighted in the dashboard** and easier for stakeholders, especially non-technical users, to interpret.

From the visualization, it was observed that sales generally declined around 2005. While SQL queries alone could not fully reveal these insights, the dashboard made it possible to

identify that the company is carrying a high level of uncollected credit risk. **The downward trend in the line chart likely reflects the impact of an unsustainable credit strategy, which is affecting the company’s ability to maintain consistent revenue growth.**

